

Category C — Assignment Problems

Problem 1

The Health Bar

Scenario

A game character has health that changes during battle.

Problem Statement

Design a Character class where health can never drop below 0 or rise above its maximum, and can't be set directly from outside.

Requirements

- Health must be private, changed only through takeDamage(int amount) and heal(int amount).
- Health must never go below 0 (extra damage is just wasted) or above the maximum (extra healing is just wasted).
- The maximum health must be final, fixed when the character is created.
- Provide a read-only way to check current health — no setter for it.

Expected Behavior

- A character with 100 max health that takes 30 damage has 70 health left.
- Healing past the maximum caps at the maximum, not above it.
- Damage that would take health below 0 leaves health at exactly 0, not negative.

Sample Input/Output

```
Character c = new Character(100);
```

```
c.takeDamage(30) -> health = 70
```

```
c.heal(50) -> health = 100 (capped)
```

```
c.takeDamage(150) -> health = 0 (floored)
```

Hints

Hint 1: Both takeDamage and heal need a boundary check — one clamps at 0, the other clamps at the maximum.

Hint 2: Think of it as "calculate the new value first, then clamp it" rather than trying to prevent the crossing in advance.

Problem 2

The Playlist

Scenario

A music app lets you build a playlist of songs.

Problem Statement

Design a Playlist class that stores songs internally but returns a safe copy of the list, so nobody can sneak in changes from outside.

Requirements

- Store song titles in a private array (assume a fixed maximum size).
- Provide a method to add a song, and one that returns all the songs added so far — as a copy, not the original array.
- Changing the array returned by that method must not affect the playlist's real contents.
- Provide a read-only count of how many songs are in the playlist.

Expected Behavior

- Adding "Song A" and "Song B" and then asking for the list returns both, in order.
- Modifying the array you got back from the playlist does not change what the playlist actually holds.
- `getSongCount()` always matches how many songs have actually been added.

Sample Input/Output

```
Playlist p = new Playlist(10);  
p.addSong("Song A"); p.addSong("Song B");  
String[] copy = p.getSongs();  
copy[0] = "Hacked"; // p.getSongs()[0] is still "Song A"
```

Hints

Hint 1: Returning your private array directly hands out a way to edit it from outside — build a brand new array with the same contents to hand back instead.

Hint 2: Java arrays have a simple way to duplicate their contents into a new array — you don't need to copy element by element by hand, though you can if you prefer.

Problem 3

The Password Checker

Scenario

A signup form checks how strong your chosen password is.

Problem Statement

Design a PasswordChecker class that accepts a password once and reveals only its strength rating — never the password itself.

Requirements

- Take the password as a string in the constructor and store it privately, with no getter that returns it.
- Provide a method that returns a strength label — for example "Weak" (under 6 characters), "Medium" (6-9 characters), or "Strong" (10+ characters, your own rule is fine).
- The password itself must never be changeable after the object is created.
- Use the String length (and any other simple check you like, such as whether it contains a digit) to decide the rating.

Expected Behavior

- A 4-character password rates "Weak".
- An 8-character password rates "Medium".
- A 12-character password rates "Strong".
- There is no method anywhere that returns the actual password text.

Sample Input/Output

```
PasswordChecker pc = new PasswordChecker("abcd");  
pc.getStrength() -> "Weak"  
PasswordChecker pc2 = new PasswordChecker("abcdefghij");  
pc2.getStrength() -> "Strong"
```

Hints

Hint 1: length() on a String gives you everything you need for the basic version of this rating.

Hint 2: Compute the rating inside getStrength() using the stored password — you never need to expose the password to do that.

Problem 4

The Traffic Light

Scenario

A traffic light cycles through red, green, and yellow, in order.

Problem Statement

Design a TrafficLight class where the color can only move forward through its cycle, never be set to an arbitrary color directly.

Requirements

- The current color must be private, changed only by a next() method that moves red -> green -> yellow -> red, in that order.
- There must be no method that sets the color directly to any value.
- Provide a read-only way to check the current color.
- Give the light a final ID, fixed when it's created.

Expected Behavior

- A new light starts on red.
- Calling next() repeatedly cycles red -> green -> yellow -> red -> green ... forever, in that exact order.
- There's no way to jump the light straight to, say, yellow without going through the proper order.

Sample Input/Output

```
TrafficLight t = new TrafficLight("TL-9");  
t.getColor() -> "RED"  
t.next() -> "GREEN"  
t.next() -> "YELLOW"  
t.next() -> "RED"
```

Hints

Hint 1: A simple set of if-checks (or a small array of the three colors in order) inside next() is enough to decide what comes after the current color.

Hint 2: The whole point is that next() is the only door in or out of that field — resist adding any other way to change it.

Problem 5

The Shopping Cart

Scenario

An online store's cart holds the prices of items you're about to buy.

Problem Statement

Design a Cart class that stores item prices internally but only exposes the total and item count — never direct access to the prices themselves.

Requirements

- Store item prices in a private array (assume a fixed maximum number of items).
- Provide a method to add an item's price to the cart.
- Provide a read-only total (sum of all prices) and a read-only item count — computed on request, not stored separately.
- Give the cart a final cart ID, fixed when it's created.

Expected Behavior

- Adding prices 250, 99, and 151 gives a total of 500 and a count of 3.
- There is no method that returns the array of individual prices.
- The total is always correct immediately after any item is added, with no extra step needed.

Sample Input/Output

```
Cart cart = new Cart("CART-5", 20);
cart.addItem(250); cart.addItem(99); cart.addItem(151);
cart.getTotal() -> 500
cart.getItemCount() -> 3
```

Hints

Hint 1: getTotal() can simply loop over the private array and add everything up each time it's called — it doesn't need a separate running-total field.

Hint 2: Keep a counter for how many items have been added so far, so you know both where the next price goes and what the count is.